

Project Topic: Neural Spectral Capacity as a Heuristic for Architecture Design

Before you start. The recommended core project uses a supplied Python starter and archived Transformer outcomes. It runs on a laptop without a GPU or LLM API. Complete one focused study; the advanced directions are optional. All submissions, deadlines and grading follow the [Course Project page](#).

1. Background

In informed search, a heuristic helps decide which states to explore. Neural architecture search faces a similar choice: fully training every candidate network is expensive. A cheap architecture score can help select promising candidates before spending a training budget.

Neural Spectral Capacity (NSC) estimates capacity from matrix dimensions and a fixed initialization convention, then adds the estimates across the architecture [1]. It is an analytic heuristic for neural networks; it is not a learned performance predictor or an admissible A* cost-to-go estimate. Whether it predicts trained performance is an empirical question.

For a matrix W with singular values σ_i , the reference capacity is

$$\psi(W) = \ln \det(I + W^T W) = \sum_i \ln(1 + \sigma_i^2).$$

The Marchenko-Pastur (MP) law approximates its expectation by $\psi_{\text{MP}}(m, n, s)$, using matrix dimensions and initialization standard deviation s . Original NSC sums this quantity over selected matrices. The starter supplies this calculation; deriving the MP law is not required.

A **computational graph** also records dependencies, residual additions, shared states and reused weights. A sum of independent matrix scores can miss these features. For example, reordering the same branches leaves an additive score unchanged.

Project question. Can one motivated change make NSC more sensitive to a computational-graph mechanism, and does that change improve the ranking or selection of trained architectures under a fixed resource budget?

These are two separate claims: distinguishing graphs does not by itself demonstrate better prediction. A well-supported negative result is a valid project outcome.

2. Provided starting experiment

Download [nsc-starter.zip](#) and start with its [README](#). Python 3.9 or later is sufficient; there are no third-party dependencies. Basic Python, matrix operations and familiarity with attention, FFNs and residual connections are useful prerequisites. The core task does not require implementing or training a Transformer.

- **Architecture panel:** 200 archived decoder-Transformer configurations used in NSC's GPT-2/LiteTransformerSearch evaluation, with reported parameters and trained perplexity. Width, depth, head counts and FFN allocations vary. A fixed split supplies 100 development architectures and 100 final architectures.
- **Reference scorer and resources:** `nsc.py` implements original NSC, a simple positional example, projection-parameter counts and a stated reference FLOP estimate.
- **Evaluation:** `evaluate.py` compares ranking and fixed-budget selection. `student_score.py` is the interface for your score and its ablation. Numerical checks and expected development output are included.

```
python3 check_starter.py
python3 evaluate.py --split development --output dev.json
```

Scope of the data. The archive contains memory and adaptive-embedding settings; inspect its configurations instead of assuming a modern GPT block. It records one outcome per architecture and lacks seed-level training records and a complete training protocol. Use it for an exploratory comparison on this fixed panel. It contains no trained labels for new MoE, loop, KV-sharing or hyperconnection variants, or for the synthetic reorder pair below.

2.1 Original score and resource convention

For hidden width d , layer head count H and FFN width f , the supplied baseline uses $s = 0.02$ and scores Q/K/V per head, one attention output projection and two FFN projections:

$$S_0 = \sum_{\text{layers}} [3H\psi_{\text{MP}}(d, d/H, s) + \psi_{\text{MP}}(d, d, s) + 2\psi_{\text{MP}}(d, f, s)].$$

Keep initialization and grouping fixed in comparisons. This score excludes embeddings, biases, normalization and the vocabulary head. Selection budgets use the archive's reported **total parameters**; the simplified FLOP estimate counts projections and dense attention products at a fixed key length. Its omissions are documented in the README. Neither quantity is measured latency.

2.2 Worked example: sensitivity is not prediction

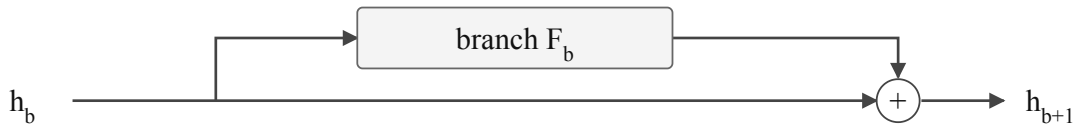
Consider two layers with $d = 128$, four attention heads each, and FFN widths 256 and 512. Reverse the FFN allocation. Both designs have 327,680 projection weights and 524,288 reference MACs per token. Original NSC ties. The supplied teaching example weights layer l 's capacity by l/L :

FFN widths, first to last	Original NSC	Positional example
[256, 512]	121.820170	96.957958
[512, 256]	121.820170	85.772297

The new scores differ, but there are no trained outcomes for this pair. The weighting demonstrates an interface and an ablation, not a validated capacity theory. Your project must justify its mechanism and separately assess predictive usefulness.

3. Recommended core project: Transformer graph fidelity

Study **one** mechanism using the supplied dense-Transformer panel. For example, investigate how layer position and FFN allocation interact with residual context, or how a branch's projections should be aggregated. A motivated heuristic modification is sufficient; a new closed-form theory is not required.



A residual update. Specify normalization and branch assumptions in your reference model; this sketch does not prescribe a pre- or post-normalization architecture.

1. **Reproduce the starting point.** Run the checks and development evaluation. Confirm the original score, resource convention and toy example. The unedited student template returns original NSC.
2. **Develop one change.** Define the graph feature, scored units, assumptions and aggregation rule. Implement a specification-only score and an ablation that removes the new mechanism. Include a controlled example and a limiting-case or numerical check. Merely using the supplied positional example is insufficient.
3. **Evaluate ranking.** Compare with original NSC, the positional example, the ablation, reported parameter count and reference FLOPs. Report Spearman's ρ and Kendall's τ_b against **negative** perplexity, plus ties. Include the supplied size-controlled comparisons: same hidden width and total parameters within 10%. Report how many comparisons remain; this is not a causal control.
4. **Evaluate selection.** Within each supplied parameter cap (10M, 20M, 30M, 50M), select $k = 1$ and 3 candidates using each heuristic and uniform random selection. Use the same pool and k . Reveal their archived outcomes; report best selected perplexity and regret relative to the best feasible candidate. Enumeration and sorting are sufficient; a new search algorithm is optional.
5. **Freeze and assess.** Develop on the development split, then freeze your method and choices before using `--split final`. Discuss failures, resource effects and scoring cost. Distinguish random-selection variability from training-seed uncertainty, which this archive cannot estimate.

What counts as a contribution? A justified modification or substantive controlled study, correct implementation, ablation and evidence about when it helps or fails. Cite the published methods you build on and distinguish reproduction from your own contribution. The supplied positional example is a teaching illustration. Publication-level novelty or outperforming NSC is not required.

4. Potential extensions

The core study above is a complete project scope. The following retain the broader research directions for groups with the relevant background and resources. Choose at most one extension. State the evaluation scope in your proposal; new architecture families need corresponding trained outcomes for performance claims. Synthetic or linear experiments can validate a reference model, but cannot establish Transformer performance.

Direction 1. Stronger Transformer-block modeling

Investigate whether accounting for normalization, residual connections or dependencies within a branch improves the heuristic. Choose one mechanism, state your assumptions and compare with an ablation that removes it. Controlled trained reorderings would provide stronger evidence than the archive's heterogeneous configurations. Time a small training run before planning a new panel.

Direction 2. Mixture-of-experts (MoE)

Develop a heuristic for architectures that store multiple experts but activate only a subset for each token, under explicit routing assumptions [3]. Compare all-expert and selected-expert sums, then investigate gate weights, shared experts or residual placement. Count the router, stored parameters and active compute separately. Start within one MoE family; a common dense/MoE score scale needs justification.

Direction 3. Attention state and the KV cache

Study shared K/V heads in grouped-query attention (GQA) [4], or compressed latent state in MLA [5]. Count shared projections once, define how sharing changes the score, and compare head or latent-dimension allocations under a KV-memory budget. Fix sequence length, batch size, precision and inference regime; derive memory from the tensors actually retained.

Direction 4. Looped blocks and parameter reuse

Compare repeated execution of one block with independent layers [6]. Use unique-parameter and invocation-counted sums as baselines for a reuse-aware model. Tied weights remain the same across passes; an independence assumption changes the reference problem. Report stored parameters and execution compute separately. Use a small fixed menu of loop counts.

Direction 5. Hyperconnections and multiple residual streams

Model read, write and mixing operations across multiple residual streams [7] [8]. Begin with fixed mixing and compare identity, permutation and rank-deficient cases. A stream permutation is a useful invariance check; simply multiplying by stream count is not evidence of improved modeling. Include stream memory and mixing costs. Dynamic mixing and combined loops are more demanding extensions.

5. What to hand in

Follow the course's one-page proposal, milestone and four-page final-report rules; this topic adds no separate report or grading scheme. In the proposal, identify your mechanism, data, baselines and compute plan. By the milestone, have the baseline, your score and preliminary evaluation running.

In the final report, explain the mechanism and assumptions; show a controlled check, ranking and selection results, an ablation, and limitations. Submit reproducible code, configurations, exact commands and the graded commit/tag. Full output tables may live in the repository; the report should explain the findings. Careful analysis of a failed hypothesis is preferable to unsupported claims of improvement.

6. Useful resources

1. **NSC:** Zhu, Zhao and Lu. *Neural Spectral Capacity: Measuring and Designing Architectures from Network Specification Alone*. [Paper](#); [official code](#). Start with the course starter's documented conventions.
2. **Archived panel attribution:** Javaheripi et al. *LiteTransformerSearch: Training-free Neural Architecture Search for Efficient Language Models*. NeurIPS 2022. [Paper](#); [course-data provenance and limitations](#).
3. **MoE:** Fedus et al. *Switch Transformers*. [Paper](#).
4. **Shared KV:** Ainslie et al. *GQA*. EMNLP 2023. [Paper](#).
5. **MLA:** DeepSeek-AI. *DeepSeek-V2*. [Paper](#).
6. **Loops:** Zeitoun et al. *Hyperloop Transformers*. [Paper](#).
7. **Multiple streams:** Zhu et al. *Hyper-Connections*. [Paper](#).
8. **Constrained mixing:** Xie et al. *mHC: Manifold-Constrained Hyper-Connections*. [Paper](#).